

 meta-llama/

Meta-Llama-3.1-8B-Instruct



Public



\$0.02 in, \$0.05 out / 1M



bfloat16



131,072



JSON



Function

OpenAI Chat Completions

Python

OpenAI with python client

You can use the official openai python client to run inferences with us

```
# Assume openai>=1.0.0
from openai import OpenAI

# Create an OpenAI client with your deepinfra token and endpoint
openai = OpenAI(
    api_key="u7u0yGoZ6CA61tYm3wfiTliyJn9kvAli",
    base_url="https://api.deepinfra.com/v1/openai",
)

chat_completion = openai.chat.completions.create(
    model="meta-llama/Meta-Llama-3.1-8B-Instruct",
    messages=[{"role": "user", "content": "Hello"}],
)

print(chat_completion.choices[0].message.content)
print(chat_completion.usage.prompt_tokens,
      chat_completion.usage.completion_tokens)

# Hello! It's nice to meet you. Is there something I can help you with, or would
```

copy

```
you like to chat?  
# 11 25
```

Conversations

To create a longer chat-like conversation you just have to add each response message and each of the user messages to every request. This way the model will have the context and will be able to provide better answers. You can tweak it even further by providing a system message.

```
# Assume openai>=1.0.0 copy  
from openai import OpenAI  
  
# Create an OpenAI client with your deepinfra token and endpoint  
openai = OpenAI(  
    api_key="u7u0yGoZ6CA61tYm3wfiTliyJn9kvAli",  
    base_url="https://api.deepinfra.com/v1/openai",  
)  
  
chat_completion = openai.chat.completions.create(  
    model="meta-llama/Meta-Llama-3.1-8B-Instruct",  
    messages=[  
        {"role": "system", "content": "Respond like a michelin starred chef."},  
        {"role": "user", "content": "Can you name at least two different  
techniques to cook lamb?"},  
        {"role": "assistant", "content": "Bonjour! Let me tell you, my friend,  
cooking lamb is an art form, and I'm more than happy to share with you not two,  
but three of my favorite techniques to coax out the rich, unctuous flavors and  
tender textures of this majestic protein. First, we have the classic \"Sous  
Vide\" method. Next, we have the ancient art of \"Sous le Sable\". And finally,  
we have the more modern technique of \"Hot Smoking.\""},  
        {"role": "user", "content": "Tell me more about the second method."},  
    ],  
)  
  
print(chat_completion.choices[0].message.content)  
print(chat_completion.usage.prompt_tokens,  
      chat_completion.usage.completion_tokens)  
  
# Sous le Sable! It's an ancient technique that never goes out of style, n'est-  
ce pas? Literally ...  
# 149 324
```

The longer the conversation gets, the more time it takes the model to generate the response. The number of messages that you can have in a conversation is limited by the context size of a model. Larger models also usually take more time to respond.

Streaming

Streaming any of the chat completions above is supported by adding the `stream=True` option.

```
from openai import OpenAIcopy

# Create an OpenAI client with your deepinfra token and endpoint
openai = OpenAI(
    api_key="u7u0yGoZ6CA61tYm3wfiTliyJn9kvAli",
    base_url="https://api.deepinfra.com/v1/openai",
)

chat_completion = openai.chat.completions.create(
    model="meta-llama/Meta-Llama-3.1-8B-Instruct",
    messages=[{"role": "user", "content": "Hello"}],
    stream=True,
)

for event in chat_completion:
    if event.choices[0].finish_reason:
        print(event.choices[0].finish_reason, event.usage["prompt_tokens"],
event.usage["completion_tokens"])
    else:
        print(event.choices[0].delta.content)

# Hello
# !
# It
# 's
# nice
# ...
# 11 25
```

Input fields

`model` string

model name

`messages` array

conversation messages: (user,assistant,tool)*,user including one system message anywhere

`stream` boolean

whether to stream the output via SSE or return the full response

Default value: `false`

`temperature` number

What sampling temperature to use, between 0 and 2. Higher values like 0.8 will make the output more random, while lower values like 0.2 will make it more focused and deterministic

Default value: `1`

Range: `0 ≤ temperature ≤ 2`

`top_p` number

An alternative to sampling with temperature, called nucleus sampling, where the model considers the results of the tokens with top_p probability mass. So 0.1 means only the tokens comprising the top 10% probability mass are considered.

Default value: `1`

Range: `0 < top_p ≤ 1`

`min_p` number

Float that represents the minimum probability for a token to be considered, relative to the probability of the most likely token. Must be in [0, 1]. Set to 0 to disable this.

Default value:

Range:

integer

Sample from the best k (number of) tokens. 0 means off

Default value:

Range:

integer

The maximum number of tokens to generate in the chat completion. The total length of input tokens and generated tokens is limited by the model's context length. If explicitly set to None it will be the model's max context length minus input length or 16384, whichever is smaller.

Range:

string

up to 16 sequences where the API will stop generating further tokens

array

Up to 16 token IDs where the API will stop generating further tokens. Merged with the model's built-in stop tokens. Intended for private deployments.

integer

number of sequences to return

Default value:

Range:

`presence_penalty` number

Positive values penalize new tokens based on whether they appear in the text so far, increasing the model's likelihood to talk about new topics.

Default value: `0`

Range: $-2 \leq \text{presence_penalty} \leq 2$

`frequency_penalty` number

Positive values penalize new tokens based on how many times they appear in the text so far, increasing the model's likelihood to talk about new topics.

Default value: `0`

Range: $-2 \leq \text{frequency_penalty} \leq 2$

`tools` array

A list of tools the model may call. Currently, only functions are supported as a tool.

`tool_choice` string

Controls which (if any) function is called by the model. `none` means the model will not call a function and instead generates a message. `auto` means the model can pick between generating a message or calling a function. `required` means the model must call a function. `defined tool` means the model must call that specific tool. `none` is the default when no functions are present. `auto` is the default if functions are present.

`response_format` any

The format of the response. Currently, only json is supported.

`repetition_penalty` number

Alternative penalty for repetition, but multiplicative instead of additive (> 1 penalize, < 1 encourage)

Default value:

1

Range:

$0.01 \leq \text{repetition_penalty} \leq 5$

user string

A unique identifier representing your end-user, which can help monitor and detect abuse. Avoid sending us any identifying information. We recommend hashing user identifiers.

seed integer

Seed for random number generator. If not provided, a random seed is used. Determinism is not guaranteed.

Range:

$-9223372036854776000 \leq \text{seed} < 18446744073709552000$

logprobs boolean

Whether to return log probabilities of the output tokens or not. If true, returns the log probabilities of each output token returned in the `content` of `message`.

stream_options object

streaming options

reasoning_effort string

Constrains effort on reasoning for reasoning models. Currently supported values are none, low, medium, high, and xhigh. Reducing reasoning effort can result in faster responses and fewer tokens used on reasoning in a response. Setting to none disables reasoning entirely if the model supports.

Allowed values:

low

medium

high

xhigh

none

reasoning object

Reasoning configuration.

`prompt_cache_key` string

A key to identify prompt cache for reuse across requests. If provided, the prompt will be cached and can be reused in subsequent requests with the same key.

`chat_template_kwargs` object

Chat template kwargs.

`continue_final_message` boolean

If set, the final assistant message is used as a prefix for the model to continue generating from, rather than starting a new turn. Only applicable when the last message in the conversation is an assistant message.

Input Schema

This is the detailed description of the input parameters in JSON Schema format

```
{
  "definitions": {
    "ChatCompletionAssistantMessage": {
      "properties": {
        "cache_control": {
          "additionalProperties": true,
          "description": "Cache control for prompt caching",
          "title": "Cache Control",
          "type": "object"
        },
        "role": {
          "const": "assistant",
          "default": "assistant",
          "description": "the role of the author of this message",
          "title": "Role",
          "type": "string"
        },
        "content": {
          "anyOf": [
```



```

        {
            "type": "string"
        },
        {
            "items": {
                "$ref":
"#/definitions/ChatCompletionContentPartText"
            },
            "type": "array"
        }
    ],
    "description": "the message content",
    "title": "Content"
},
"reasoning_content": {
    "description": "the reasoning content",
    "title": "Reasoning Content",
    "type": "string"
},
"name": {
    "title": "Name",
    "type": "string"
},
"tool_calls": {
    "description": "the tool calls generated by the mode",
    "items": {
        "$ref": "#/definitions/ChatCompletionMessageToolCall"
    },
    "title": "Tool Calls",
    "type": "array"
}
},
"title": "ChatCompletionAssistantMessage",
"type": "object"
},
"ChatCompletionContentPartAudio": {
    "properties": {
        "type": {
            "const": "input_audio",
            "title": "Type",
            "type": "string"
        },
        "input_audio": {
            "$ref": "#/definitions/InputAudio"
        }
    }
},

```

```

    "required": [
        "type",
        "input_audio"
    ],
    "title": "ChatCompletionContentPartAudio",
    "type": "object"
},
"ChatCompletionContentPartImage": {
    "properties": {
        "type": {
            "const": "image_url",
            "title": "Type",
            "type": "string"
        },
        "image_url": {
            "$ref": "#/definitions/ImageURL"
        }
    },
    "required": [
        "type",
        "image_url"
    ],
    "title": "ChatCompletionContentPartImage",
    "type": "object"
},
"ChatCompletionContentPartText": {
    "properties": {
        "type": {
            "const": "text",
            "title": "Type",
            "type": "string"
        },
        "text": {
            "title": "Text",
            "type": "string"
        }
    },
    "required": [
        "type",
        "text"
    ],
    "title": "ChatCompletionContentPartText",
    "type": "object"
},
"ChatCompletionContentPartVideo": {
    "properties": {

```

```

        "type": {
            "const": "video_url",
            "title": "Type",
            "type": "string"
        },
        "video_url": {
            "$ref": "#/definitions/VideoURL"
        }
    },
    "required": [
        "type",
        "video_url"
    ],
    "title": "ChatCompletionContentPartVideo",
    "type": "object"
},
"ChatCompletionMessageToolCall": {
    "properties": {
        "id": {
            "description": "the id of the tool call",
            "title": "Id",
            "type": "string"
        },
        "type": {
            "description": "the type of the tool call. only function is supported currently",
            "title": "Type",
            "type": "string"
        },
        "function": {
            "$ref": "#/definitions/Function",
            "description": "the function that the model called"
        }
    },
    "required": [
        "id",
        "type",
        "function"
    ],
    "title": "ChatCompletionMessageToolCall",
    "type": "object"
},
"ChatCompletionSystemMessage": {
    "properties": {
        "cache_control": {
            "additionalProperties": true,

```

```

        "description": "Cache control for prompt caching",
        "title": "Cache Control",
        "type": "object"
    },
    "role": {
        "const": "system",
        "default": "system",
        "description": "the role of the author of this message",
        "title": "Role",
        "type": "string"
    },
    "content": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "items": {
                    "$ref":
"#/definitions/ChatCompletionContentPartText"
                },
                "type": "array"
            }
        ],
        "description": "the message content",
        "title": "Content"
    },
    "name": {
        "title": "Name",
        "type": "string"
    }
},
"required": [
    "content"
],
"title": "ChatCompletionSystemMessage",
"type": "object"
},
"ChatCompletionToolMessage": {
    "properties": {
        "cache_control": {
            "additionalProperties": true,
            "description": "Cache control for prompt caching",
            "title": "Cache Control",
            "type": "object"
        },

```

```

    "role": {
      "const": "tool",
      "default": "tool",
      "description": "the role of the author of this message",
      "title": "Role",
      "type": "string"
    },
    "content": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "items": {
            "$ref":
"#/definitions/ChatCompletionContentPartText"
          },
          "type": "array"
        }
      ],
      "description": "the message content",
      "title": "Content"
    },
    "tool_call_id": {
      "title": "Tool Call Id",
      "type": "string"
    }
  },
  "required": [
    "content",
    "tool_call_id"
  ],
  "title": "ChatCompletionToolMessage",
  "type": "object"
},
"ChatCompletionUserMessage": {
  "properties": {
    "cache_control": {
      "additionalProperties": true,
      "description": "Cache control for prompt caching",
      "title": "Cache Control",
      "type": "object"
    },
    "role": {
      "const": "user",
      "default": "user",

```

```

        "description": "the role of the author of this message",
        "title": "Role",
        "type": "string"
    },
    "content": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "items": {
                    "anyOf": [
                        {
                            "$ref":
"#/definitions/ChatCompletionContentPartText"
                        },
                        {
                            "$ref":
"#/definitions/ChatCompletionContentPartImage"
                        },
                        {
                            "$ref":
"#/definitions/ChatCompletionContentPartAudio"
                        },
                        {
                            "$ref":
"#/definitions/ChatCompletionContentPartVideo"
                        }
                    ]
                },
                "type": "array"
            }
        ],
        "description": "the message content",
        "title": "Content"
    },
    "name": {
        "title": "Name",
        "type": "string"
    }
},
"required": [
    "content"
],
"title": "ChatCompletionUserMessage",
"type": "object"

```

```

    },
    "ChatReasoningSettings": {
      "properties": {
        "effort": {
          "description": "Constrains effort on reasoning for reasoning
models.",
          "enum": [
            "low",
            "medium",
            "high",
            "xhigh"
          ],
          "title": "Effort",
          "type": "string"
        },
        "enabled": {
          "description": "Enable or disable reasoning with default
parameters.",
          "title": "Enabled",
          "type": "boolean"
        }
      },
      "title": "ChatReasoningSettings",
      "type": "object"
    },
    "ChatTools": {
      "properties": {
        "cache_control": {
          "additionalProperties": true,
          "description": "Cache control for prompt caching",
          "title": "Cache Control",
          "type": "object"
        },
        "type": {
          "default": "function",
          "title": "Type",
          "type": "string"
        },
        "function": {
          "$ref": "#/definitions/FunctionDefinition"
        }
      },
      "required": [
        "function"
      ],
      "title": "ChatTools",

```

```

        "type": "object"
    },
    "Function": {
        "properties": {
            "name": {
                "description": "the name of the function to call",
                "title": "Name",
                "type": "string"
            },
            "arguments": {
                "description": "the function arguments, generated by the
model in JSON format. the model does not always generate valid JSON, and may
hallucinate parameters not defined by your function schema",
                "title": "Arguments",
                "type": "string"
            }
        },
        "required": [
            "name",
            "arguments"
        ],
        "title": "Function",
        "type": "object"
    },
    "FunctionDefinition": {
        "properties": {
            "name": {
                "title": "Name",
                "type": "string"
            },
            "description": {
                "title": "Description",
                "type": "string"
            },
            "parameters": {
                "additionalProperties": true,
                "title": "Parameters",
                "type": "object"
            }
        },
        "required": [
            "name"
        ],
        "title": "FunctionDefinition",
        "type": "object"
    },

```



```
"ImageUrl": {
  "properties": {
    "url": {
      "title": "Url",
      "type": "string"
    },
    "detail": {
      "default": "auto",
      "enum": [
        "auto",
        "low",
        "high"
      ],
      "title": "Detail",
      "type": "string"
    }
  },
  "required": [
    "url"
  ],
  "title": "ImageUrl",
  "type": "object"
},
"InputAudio": {
  "properties": {
    "data": {
      "title": "Data",
      "type": "string"
    },
    "format": {
      "default": "wav",
      "enum": [
        "wav",
        "mp3"
      ],
      "title": "Format",
      "type": "string"
    }
  },
  "required": [
    "data"
  ],
  "title": "InputAudio",
  "type": "object"
},
"JsonObjectResponseFormat": {
```

```

    "properties": {
      "type": {
        "const": "json_object",
        "default": "json_object",
        "title": "Type",
        "type": "string"
      }
    },
    "title": "JsonObjectResponseFormat",
    "type": "object"
  },
  "JsonSchema": {
    "properties": {
      "name": {
        "description": "Name identifier for the JSON schema",
        "title": "Name",
        "type": "string"
      },
      "schema": {
        "additionalProperties": true,
        "description": "The actual JSON schema definition",
        "title": "Schema",
        "type": "object"
      }
    },
    "required": [
      "name",
      "schema"
    ],
    "title": "JsonSchema",
    "type": "object"
  },
  "JsonSchemaResponseFormat": {
    "properties": {
      "type": {
        "const": "json_schema",
        "default": "json_schema",
        "title": "Type",
        "type": "string"
      },
      "json_schema": {
        "$ref": "#/definitions/JsonSchema",
        "description": "JSON schema for structured output when type
is 'json_schema'"
      }
    },
  },

```

```

    "required": [
        "json_schema"
    ],
    "title": "JsonSchemaResponseFormat",
    "type": "object"
},
"RegexResponseFormat": {
    "properties": {
        "type": {
            "const": "regex",
            "default": "regex",
            "title": "Type",
            "type": "string"
        },
        "regex": {
            "description": "Regex pattern for structured output when
type is 'regex'",
            "title": "Regex",
            "type": "string"
        }
    },
    "required": [
        "regex"
    ],
    "title": "RegexResponseFormat",
    "type": "object"
},
"StreamOptions": {
    "properties": {
        "include_usage": {
            "default": true,
            "description": "whether to include usage data",
            "title": "Include Usage",
            "type": "boolean"
        },
        "continuous_usage_stats": {
            "default": false,
            "description": "whether to include usage stats continuously
with each streaming event",
            "title": "Continuous Usage Stats",
            "type": "boolean"
        }
    },
    "title": "StreamOptions",
    "type": "object"
},

```

```

    "TextResponseFormat": {
      "properties": {
        "type": {
          "const": "text",
          "default": "text",
          "title": "Type",
          "type": "string"
        }
      },
      "title": "TextResponseFormat",
      "type": "object"
    },
    "VideoURL": {
      "properties": {
        "url": {
          "title": "Url",
          "type": "string"
        }
      },
      "required": [
        "url"
      ],
      "title": "VideoURL",
      "type": "object"
    }
  ],
  "required": [
    "model",
    "messages"
  ],
  "title": "OpenAIChatCompletionsIn",
  "type": "object",
  "properties": {
    "model": {
      "description": "model name",
      "title": "Model",
      "type": "string",
      "example": "meta-llama/Llama-2-70b-chat-hf"
    },
    "messages": {
      "description": "conversation messages: (user,assistant,tool)*,user including one system message anywhere",
      "items": {
        "anyOf": [
          {
            "$ref": "#/definitions/ChatCompletionToolMessage"

```

```

        },
        {
            "$ref": "#/definitions/ChatCompletionAssistantMessage"
        },
        {
            "$ref": "#/definitions/ChatCompletionUserMessage"
        },
        {
            "$ref": "#/definitions/ChatCompletionSystemMessage"
        }
    ]
},
"title": "Messages",
"type": "array"
},
"stream": {
    "default": false,
    "description": "whether to stream the output via SSE or return the
full response",
    "title": "Stream",
    "type": "boolean"
},
"temperature": {
    "default": 1,
    "description": "What sampling temperature to use, between 0 and 2.
Higher values like 0.8 will make the output more random, while lower values like
0.2 will make it more focused and deterministic",
    "maximum": 2,
    "minimum": 0,
    "title": "Temperature",
    "type": "number"
},
"top_p": {
    "default": 1,
    "description": "An alternative to sampling with temperature, called
nucleus sampling, where the model considers the results of the tokens with top_p
probability mass. So 0.1 means only the tokens comprising the top 10%
probability mass are considered.",
    "exclusiveMinimum": 0,
    "maximum": 1,
    "title": "Top P",
    "type": "number"
},
"min_p": {
    "default": 0,
    "description": "Float that represents the minimum probability for a

```

token to be considered, relative to the probability of the most likely token. Must be in [0, 1]. Set to 0 to disable this.",

```
    "maximum": 1,
    "minimum": 0,
    "title": "Min P",
    "type": "number"
  },
  "top_k": {
    "default": 0,
    "description": "Sample from the best k (number of) tokens. 0 means
off",
    "exclusiveMaximum": 1000,
    "minimum": 0,
    "title": "Top K",
    "type": "integer"
  },
  "max_tokens": {
    "description": "The maximum number of tokens to generate in the chat
completion.\n\nThe total length of input tokens and generated tokens is limited
by the model's context length. If explicitly set to None it will be the model's
max context length minus input length or 16384, whichever is smaller.",
    "maximum": 1000000,
    "minimum": 0,
    "title": "Max Tokens",
    "type": "integer"
  },
  "stop": {
    "anyOf": [
      {
        "type": "string"
      },
      {
        "items": {
          "type": "string"
        },
        "type": "array"
      }
    ],
    "description": "up to 16 sequences where the API will stop
generating further tokens",
    "title": "Stop"
  },
  "stop_token_ids": {
    "description": "Up to 16 token IDs where the API will stop
generating further tokens. Merged with the model's built-in stop tokens.
Intended for private deployments.",
```

```

    "items": {
      "type": "integer"
    },
    "title": "Stop Token Ids",
    "type": "array"
  },
  "n": {
    "default": 1,
    "description": "number of sequences to return",
    "maximum": 4,
    "minimum": 1,
    "title": "N",
    "type": "integer"
  },
  "presence_penalty": {
    "default": 0,
    "description": "Positive values penalize new tokens based on whether
they appear in the text so far, increasing the model's likelihood to talk about
new topics.",
    "maximum": 2,
    "minimum": -2,
    "title": "Presence Penalty",
    "type": "number"
  },
  "frequency_penalty": {
    "default": 0,
    "description": "Positive values penalize new tokens based on how
many times they appear in the text so far, increasing the model's likelihood to
talk about new topics.",
    "maximum": 2,
    "minimum": -2,
    "title": "Frequency Penalty",
    "type": "number"
  },
  "tools": {
    "description": "A list of tools the model may call. Currently, only
functions are supported as a tool.",
    "items": {
      "$ref": "#/definitions/ChatTools"
    },
    "title": "Tools",
    "type": "array"
  },
  "tool_choice": {
    "anyOf": [
      {

```

```

        "type": "string"
      },
      {
        "$ref": "#/definitions/ChatTools"
      }
    ],
    "description": "Controls which (if any) function is called by the
model. none means the model will not call a function and instead generates a
message. auto means the model can pick between generating a message or calling a
function. required means the model must call a function. defined tool means the
model must call that specific tool. none is the default when no functions are
present. auto is the default if functions are present.",
    "title": "Tool Choice"
  },
  "response_format": {
    "anyOf": [
      {
        "$ref": "#/definitions/TextResponseFormat"
      },
      {
        "$ref": "#/definitions/JsonObjectResponseFormat"
      },
      {
        "$ref": "#/definitions/JsonSchemaResponseFormat"
      },
      {
        "$ref": "#/definitions/RegexResponseFormat"
      }
    ],
    "description": "The format of the response. Currently, only json is
supported.",
    "title": "Response Format"
  },
  "repetition_penalty": {
    "default": 1,
    "description": "Alternative penalty for repetition, but
multiplicative instead of additive (> 1 penalize, < 1 encourage)",
    "maximum": 5,
    "minimum": 0.01,
    "title": "Repetition Penalty",
    "type": "number"
  },
  "user": {
    "description": "A unique identifier representing your end-user,
which can help monitor and detect abuse. Avoid sending us any identifying
information. We recommend hashing user identifiers.",

```



```
    "title": "User",
    "type": "string"
  },
  "seed": {
    "description": "Seed for random number generator. If not provided, a
random seed is used. Determinism is not guaranteed.",
    "exclusiveMaximum": 18446744073709552000,
    "minimum": -9223372036854776000,
    "title": "Seed",
    "type": "integer"
  },
  "logprobs": {
    "description": "Whether to return log probabilities of the output
tokens or not.If true, returns the log probabilities of each output token
returned in the `content` of `message`.",
    "title": "Logprobs",
    "type": "boolean"
  },
  "stream_options": {
    "$ref": "#/definitions/StreamOptions",
    "description": "streaming options",
    "type": "object",
    "title": "StreamOptions"
  },
  "reasoning_effort": {
    "description": "Constrains effort on reasoning for reasoning models.
Currently supported values are none, low, medium, high, and xhigh. Reducing
reasoning effort can result in faster responses and fewer tokens used on
reasoning in a response. Setting to none disables reasoning entirely if the
model supports.",
    "enum": [
      "low",
      "medium",
      "high",
      "xhigh",
      "none"
    ],
    "title": "Reasoning Effort",
    "type": "string"
  },
  "reasoning": {
    "$ref": "#/definitions/ChatReasoningSettings",
    "description": "Reasoning configuration.",
    "type": "object",
    "title": "ChatReasoningSettings"
  },
}
```

```

    "prompt_cache_key": {
      "description": "A key to identify prompt cache for reuse across
requests. If provided, the prompt will be cached and can be reused in subsequent
requests with the same key.",
      "title": "Prompt Cache Key",
      "type": "string"
    },
    "chat_template_kwargs": {
      "additionalProperties": true,
      "description": "Chat template kwargs.",
      "title": "Chat Template Kwargs",
      "type": "object"
    },
    "continue_final_message": {
      "description": "If set, the final assistant message is used as a
prefix for the model to continue generating from, rather than starting a new
turn. Only applicable when the last message in the conversation is an assistant
message.",
      "title": "Continue Final Message",
      "type": "boolean"
    }
  }
}

```

Output Schema

This is the detailed description of the output parameters in JSON Schema format

```

{
  "definitions": {
    "ChatCompletionAssistantMessage": {
      "properties": {
        "cache_control": {
          "additionalProperties": true,
          "description": "Cache control for prompt caching",
          "title": "Cache Control",
          "type": "object"
        },
        "role": {
          "const": "assistant",
          "default": "assistant",

```

```

        "description": "the role of the author of this message",
        "title": "Role",
        "type": "string"
    },
    "content": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "items": {
                    "$ref":
"#/definitions/ChatCompletionContentPartText"
                },
                "type": "array"
            }
        ],
        "description": "the message content",
        "title": "Content"
    },
    "reasoning_content": {
        "description": "the reasoning content",
        "title": "Reasoning Content",
        "type": "string"
    },
    "name": {
        "title": "Name",
        "type": "string"
    },
    "tool_calls": {
        "description": "the tool calls generated by the mode",
        "items": {
            "$ref": "#/definitions/ChatCompletionMessageToolCall"
        },
        "title": "Tool Calls",
        "type": "array"
    }
},
"title": "ChatCompletionAssistantMessage",
"type": "object"
},
"ChatCompletionContentPartText": {
    "properties": {
        "type": {
            "const": "text",
            "title": "Type",

```

```

        "type": "string"
    },
    "text": {
        "title": "Text",
        "type": "string"
    }
},
"required": [
    "type",
    "text"
],
"title": "ChatCompletionContentPartText",
"type": "object"
},
"ChatCompletionMessageToolCall": {
    "properties": {
        "id": {
            "description": "the id of the tool call",
            "title": "Id",
            "type": "string"
        },
        "type": {
            "description": "the type of the tool call. only function is
supported currently",
            "title": "Type",
            "type": "string"
        },
        "function": {
            "$ref": "#/definitions/Function",
            "description": "the function that the model called"
        }
    },
    "required": [
        "id",
        "type",
        "function"
    ],
    "title": "ChatCompletionMessageToolCall",
    "type": "object"
},
"FinishReason": {
    "enum": [
        "stop",
        "length",
        "tool_calls",
        "content_filter",

```

```

        "malformed_function_call"
    ],
    "title": "FinishReason",
    "type": "string"
},
"Function": {
    "properties": {
        "name": {
            "description": "the name of the function to call",
            "title": "Name",
            "type": "string"
        },
        "arguments": {
            "description": "the function arguments, generated by the
model in JSON format. the model does not always generate valid JSON, and may
hallucinate parameters not defined by your function schema",
            "title": "Arguments",
            "type": "string"
        }
    },
    "required": [
        "name",
        "arguments"
    ],
    "title": "Function",
    "type": "object"
},
"OpenAIChatCompletionChoice": {
    "properties": {
        "index": {
            "description": "index of the choice in th list of choices",
            "title": "Index",
            "type": "integer"
        },
        "message": {
            "$ref": "#/definitions/ChatCompletionAssistantMessage",
            "description": "a chat completion message generated by the
model"
        }
    },
    "finish_reason": {
        "$ref": "#/definitions/FinishReason",
        "description": "the reason the model stopped generating
tokens. stop if the model hit a natural stop point or a provided stop sequence,
length if the maximum number of tokens specified in the request was reached,
tool_calls if the model called a tool."
    }
},

```

```

        "logprobs": {
            "$ref": "#/definitions/OpenAIChoiceLogProbs",
            "description": "Log probability information for the choice."
        }
    },
    "required": [
        "index",
        "message"
    ],
    "title": "OpenAIChatCompletionChoice",
    "type": "object"
},
"OpenAIChatCompletionTokenLogprob": {
    "properties": {
        "token": {
            "description": "The token.",
            "title": "Token",
            "type": "string"
        },
        "bytes": {
            "description": "A list of integers representing the UTF-8
bytes representation of the token.",
            "items": {
                "type": "integer"
            },
            "title": "Bytes",
            "type": "array"
        },
        "logprob": {
            "description": "the log probability of the token",
            "title": "Logprob",
            "type": "number"
        }
    },
    "required": [
        "token",
        "logprob"
    ],
    "title": "OpenAIChatCompletionTokenLogprob",
    "type": "object"
},
"OpenAIChoiceLogProbs": {
    "properties": {
        "content": {
            "description": "A list of message content tokens with log
probability information.",

```

```

        "items": {
            "$ref": "#/definitions/OpenAIChatCompletionTokenLogprob"
        },
        "title": "Content",
        "type": "array"
    }
},
"title": "OpenAIChoiceLogProbs",
"type": "object"
},
"PromptTokensDetails": {
    "properties": {
        "cached_tokens": {
            "description": "number of input tokens read from cache",
            "title": "Cached Tokens",
            "type": "integer"
        },
        "cache_write_tokens": {
            "description": "number of input tokens used to create cache
entry",
            "title": "Cache Write Tokens",
            "type": "integer"
        }
    },
    "title": "PromptTokensDetails",
    "type": "object"
},
"UsageInfo": {
    "additionalProperties": true,
    "properties": {
        "prompt_tokens": {
            "default": 0,
            "description": "number of tokens in the prompt",
            "title": "Prompt Tokens",
            "type": "integer"
        },
        "total_tokens": {
            "default": 0,
            "description": "total number of tokens in the completion
(prompt + completion)",
            "title": "Total Tokens",
            "type": "integer"
        },
        "completion_tokens": {
            "default": 0,
            "description": "number of tokens generated in the

```

```

completion",
    "title": "Completion Tokens",
    "type": "integer"
  },
  "estimated_cost": {
    "description": "estimated cost of the completion in USD",
    "title": "Estimated Cost",
    "type": "number"
  },
  "prompt_tokens_details": {
    "$ref": "#/definitions/PromptTokensDetails",
    "description": "breakdown of prompt tokens usage"
  }
},
"title": "UsageInfo",
"type": "object"
}
},
"required": [
  "model",
  "choices",
  "usage"
],
"title": "OpenAIChatCompletionOut",
"type": "object",
"properties": {
  "id": {
    "description": "a unique identifier for the completion",
    "title": "Id",
    "type": "string"
  },
  "object": {
    "default": "chat.completion",
    "description": "the object type, which is always chat.completion",
    "title": "Object",
    "type": "string"
  },
  "created": {
    "description": "unix timestamp of the completion",
    "title": "Created",
    "type": "integer"
  },
  "model": {
    "description": "model name",
    "title": "Model",
    "type": "string",

```



```

        "example": "meta-llama/Llama-2-70b-chat-hf"
    },
    "choices": {
        "description": "a list of chat completion choices, can be more than
one",
        "items": {
            "$ref": "#/definitions/OpenAIChatCompletionChoice"
        },
        "title": "Choices",
        "type": "array"
    },
    "usage": {
        "$ref": "#/definitions/UsageInfo",
        "description": "usage data for the completion request",
        "type": "object",
        "title": "UsageInfo"
    }
}

```

Streaming Schema

This is the detailed description of the output stream parameters in JSON Schema format

```

{
    "definitions": {
        "ChatMessageRole": {
            "enum": [
                "system",
                "user",
                "assistant",
                "tool"
            ],
            "title": "ChatMessageRole",
            "type": "string"
        },
        "FinishReason": {
            "enum": [
                "stop",
                "length",
                "tool_calls",

```

```

        "content_filter",
        "malformed_function_call"
    ],
    "title": "FinishReason",
    "type": "string"
},
"OpenAIChatCompletionStreamChoice": {
    "properties": {
        "index": {
            "description": "index of the choice in th list of choices",
            "title": "Index",
            "type": "integer"
        },
        "delta": {
            "$ref": "#/definitions/OpenAIDeltaMessage",
            "description": "a chat completion delta generated by
streamed model responses"
        },
        "logprobs": {
            "$ref": "#/definitions/OpenAIChoiceLogProbs",
            "description": "Log probability information for the choice."
        },
        "finish_reason": {
            "$ref": "#/definitions/FinishReason",
            "description": "the reason the model stopped generating
tokens. stop if the model hit a natural stop point or a provided stop sequence,
length if the maximum number of tokens specified in the request was reached,
tool_calls if the model called a tool."
        }
    },
    "required": [
        "index",
        "delta"
    ],
    "title": "OpenAIChatCompletionStreamChoice",
    "type": "object"
},
"OpenAIChatCompletionTokenLogprob": {
    "properties": {
        "token": {
            "description": "The token.",
            "title": "Token",
            "type": "string"
        },
        "bytes": {
            "description": "A list of integers representing the UTF-8

```

```

bytes representation of the token.",
    "items": {
        "type": "integer"
    },
    "title": "Bytes",
    "type": "array"
},
"logprob": {
    "description": "the log probability of the token",
    "title": "Logprob",
    "type": "number"
}
},
"required": [
    "token",
    "logprob"
],
"title": "OpenAIChatCompletionTokenLogprob",
"type": "object"
},
"OpenAIChoiceLogProbs": {
    "properties": {
        "content": {
            "description": "A list of message content tokens with log
probability information.",
            "items": {
                "$ref": "#/definitions/OpenAIChatCompletionTokenLogprob"
            },
            "title": "Content",
            "type": "array"
        }
    },
    "title": "OpenAIChoiceLogProbs",
    "type": "object"
},
"OpenAIDeltaMessage": {
    "properties": {
        "role": {
            "$ref": "#/definitions/ChatMessageRole",
            "description": "the role of the author of this message"
        },
        "content": {
            "description": "the chunk message content",
            "title": "Content",
            "type": "string"
        }
    },

```

```

    "reasoning_content": {
      "description": "the reasoning content",
      "title": "Reasoning Content",
      "type": "string"
    },
    "tool_calls": {
      "description": "tool calls generated by the model",
      "items": {
        "$ref": "#/definitions/OpenAIDeltaToolCall"
      },
      "title": "Tool Calls",
      "type": "array"
    }
  },
  "title": "OpenAIDeltaMessage",
  "type": "object"
},
"OpenAIDeltaToolCall": {
  "properties": {
    "index": {
      "title": "Index",
      "type": "integer"
    },
    "id": {
      "title": "Id",
      "type": "string"
    },
    "function": {
      "$ref": "#/definitions/OpenAIDeltaToolCallFunction"
    },
    "type": {
      "const": "function",
      "title": "Type",
      "type": "string"
    }
  },
  "required": [
    "index"
  ],
  "title": "OpenAIDeltaToolCall",
  "type": "object"
},
"OpenAIDeltaToolCallFunction": {
  "properties": {
    "arguments": {
      "title": "Arguments",

```

```

        "type": "string"
    },
    "name": {
        "title": "Name",
        "type": "string"
    }
},
"title": "OpenAIDeltaToolCallFunction",
"type": "object"
},
"PromptTokensDetails": {
    "properties": {
        "cached_tokens": {
            "description": "number of input tokens read from cache",
            "title": "Cached Tokens",
            "type": "integer"
        },
        "cache_write_tokens": {
            "description": "number of input tokens used to create cache
entry",
            "title": "Cache Write Tokens",
            "type": "integer"
        }
    },
    "title": "PromptTokensDetails",
    "type": "object"
},
"UsageInfo": {
    "additionalProperties": true,
    "properties": {
        "prompt_tokens": {
            "default": 0,
            "description": "number of tokens in the prompt",
            "title": "Prompt Tokens",
            "type": "integer"
        },
        "total_tokens": {
            "default": 0,
            "description": "total number of tokens in the completion
(prompt + completion)",
            "title": "Total Tokens",
            "type": "integer"
        },
        "completion_tokens": {
            "default": 0,
            "description": "number of tokens generated in the

```

```

completion",
    "title": "Completion Tokens",
    "type": "integer"
  },
  "estimated_cost": {
    "description": "estimated cost of the completion in USD",
    "title": "Estimated Cost",
    "type": "number"
  },
  "prompt_tokens_details": {
    "$ref": "#/definitions/PromptTokensDetails",
    "description": "breakdown of prompt tokens usage"
  }
},
"title": "UsageInfo",
"type": "object"
}
},
"required": [
  "model",
  "choices"
],
"title": "OpenAIChatCompletionStreamOut",
"type": "object",
"properties": {
  "id": {
    "description": "a unique identifier for the completion",
    "title": "Id",
    "type": "string"
  },
  "object": {
    "default": "chat.completion.chunk",
    "description": "the object type, which is always
chat.completion.chunk",
    "title": "Object",
    "type": "string"
  },
  "created": {
    "description": "unix timestamp of the completion",
    "title": "Created",
    "type": "integer"
  },
  "model": {
    "description": "model name",
    "title": "Model",
    "type": "string",

```

```
    "example": "meta-llama/Llama-2-70b-chat-hf"
  },
  "choices": {
    "description": "a list of chat completion choices, can be more than
one",
    "items": {
      "$ref": "#/definitions/OpenAIChatCompletionStreamChoice"
    },
    "title": "Choices",
    "type": "array"
  },
  "usage": {
    "$ref": "#/definitions/UsageInfo",
    "description": "usage data about request and response",
    "type": "object",
    "title": "UsageInfo"
  }
}
```


